INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Offensive Technologies

OT Lab 1 Report

On

Understanding Assembly

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 02.04.2026

Index

Introduction……. 3

1. Preparation…… ……..4-11

2. Theory……11-13

3. Disassembly…….. 14-27

4. Reversing…….. 27-44

5. References…… 44-45

Introduction

Assembly language is the native language of a computer processor and a direct human-readable form of machine code. Its importance in offensive security, malware analysis, and exploit development is immense. This lab report has been conducted under the title "Laboratory 1: Understanding Assembly & Reverse Engineering", which is an important part of the Offensive Technologies with Elements of ML course. The main objective of this lab was to gain a deep understanding of the low-level machine operations, memory management, and stack frame structure behind high-level programming languages. Buffer overflow, stack overflow, and shellcode injection are some of the most important vulnerabilities in modern cybersecurity. To exploit or prevent such vulnerabilities, it is necessary to have a clear understanding of the register usage, memory addressing, and call stack operation of the program. Through this lab, we learned how to perform static and dynamic analysis of a compiled binary file using debugger and disassembler tools such as GDB (GNU Debugger) and Objdump. One of the main goals of this lab was to gain the skills of analyzing real-world complex binaries by avoiding decompilers and reading assembly code directly. This lab also included how to bypass or analyze security measures such as ASLR and Stack Canary. The report is mainly organized based on four main tasks. The first part discusses the preparatory work such as tools setup, 32-bit and 64-bit architecture verification, and the process of disabling ASLR for debugging. The second part discusses theoretical topics such as GOT, PLT, striped binary, and breakpoint insertion mechanism in detail. The third part analyzes the differences in function prolog, epilog, and argument passing between 32-bit and 64-bit architectures. The last part recovers the logic of unknown binary files through reverse engineering, recreates them in C language, and verifies them through PoC.

This report does not contain any installation guide or unnecessary information. Only technical analysis, formatted assembly code blocks, and PoC code as evidence are included. The main goal was to gain the skills of analyzing large binaries in real-world scenarios, which will be helpful in future exploit development and malware analysis.

Task 1: Preparation

1.a. You can use any debugger/disassembler as long as it supports the given task architecture (32bit or 64bit)

Solution:

The main objective of this task was to select a debugger and disassembler that supports 32-bit and 64-bit architectures and verify its functionality.

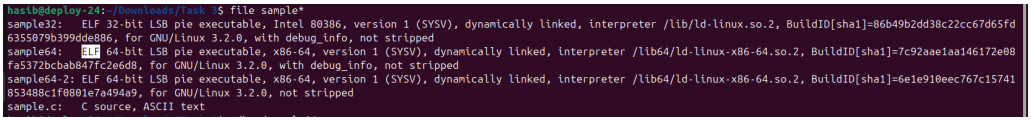
Verifying the architecture of binary files

Before starting any analysis, it is important to know what type of file it is. For this, the file command is used, which reads the *Magic Bytes* of the file, tells what architecture it is and whether it is stripped or not.

Command:

*file sample**

Output:



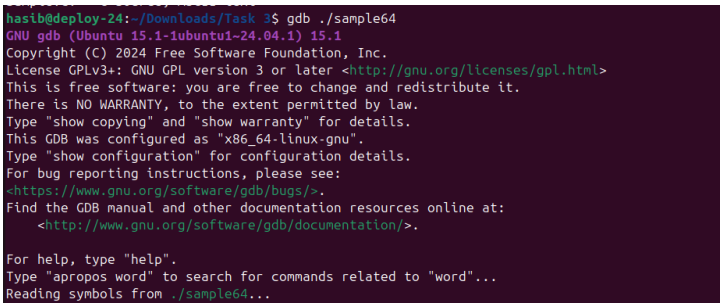
The above output confirms that both *Intel 80386 (32-bit)* and *x86-64 (64-bit)* architectures need to be worked with. Therefore, tools that support both these modes have been selected.

Debugger Selection - GDB (GNU Debugger)

GDB has been selected for dynamic analysis.

Command:

gdb ./sample64



The terminal displays the message Reading symbols from *./sample64...* . This means that *GDB* has successfully loaded the 64-bit binary and can read the debugging symbols inside it.

GDB understands both *32-bit (i386)* and *64-bit (x86_64)* instruction sets, which is ideal for this lab.

Disassembler - Using Objdump

Objdump is used to read the assembly code. It converts the machine code of the 32/64-bit binary into human-readable assembly language.

Command:

objdump -d sample64 | head -n 20

Output:

```
hasib@deploy-24:~/Downloads/Task 3$ objdump -d sample64 | head -n 20

sample64:     file format elf64-x86-64


Disassembly of section .init:

0000000000001000 <_init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    $0x8,%rsp
    1008:       48 8b 05 d9 2f 00 00    mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_start__@Base>
    100f:       48 85 c0                test   %rax,%rax
    1012:       74 02                   je     1016 <_init+0x16>
    1014:       ff d0                   call   *%rax
    1016:       48 83 c4 08             add    $0x8,%rsp
    101a:       c3                      ret

Disassembly of section .plt:

0000000000001020 <.plt>:
    1020:       ff 35 92 2f 00 00       push   0x2f92(%rip)        # 3fb8 <_GLOBAL_OFFSET_TABLE_+0x8>
```

- **file format elf64-x86-64:** The disassembler successfully recognized the *64-bit* format.
- **sub $0x8,%rsp:** Here, *%rsp* is the 64-bit stack pointer register. If it were *32-bit*, *%esp* would be seen.

Execution Permission Setup

Execute permission is required to run files in Linux.

Command:

*chmod +x sample**

```
hasib@deploy-24:~/Downloads/Task 3$ chmod +x sample*
```

The files *sample32*, *sample64*, and *sample64-2* have been made executable using *sample**.

1.b. Try to stay away from decompilers, as this will help you to better understand the nature of your task based on assembly only, remember in real-world tasks you will have to deal with much larger binaries.

(N.B. Please check what each binary does before running it – do not simply trust us!)

Solution:

The technical reasons why the manual method was chosen instead of using a direct decompiler when completing this task, and why the binary was verified before running, are outlined below.

The importance of assembly and dynamic analysis

Direct *x86_64* Assembly analysis without generating code using any automated tool.

- **Gaining Deep Knowledge:** Decompilers often simplify the underlying logic, but reading the assembly gives a clear idea of stack frames, register values, and memory addressing.
- **Practical Experience:** Real-world tasks often involve large and complex binaries that decompilers cannot handle. This approach is followed to develop the ability to extract those logics only through assembly and debugging.

"Zero-Trust" Security Policy

Pre-execution Analysis is done to ensure the security of a binary without blindly trusting it. The following 4 steps are strictly followed for each binary:

1. Identity Check: Confirming its architecture (32 or 64 bit) using the file command.

2. Embedded Data Scan: Searching for hardcoded messages or function names using strings.

3. Security Audit: Use *checksec* to check its memory protection (*NX, Canary*).

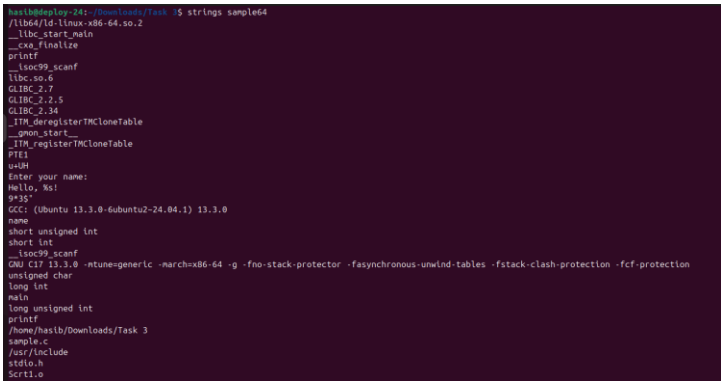
4. Controlled Execution: Run with *gdb* or *ltrace* after confirming it is safe.

Step 1: Static Analysis - strings command

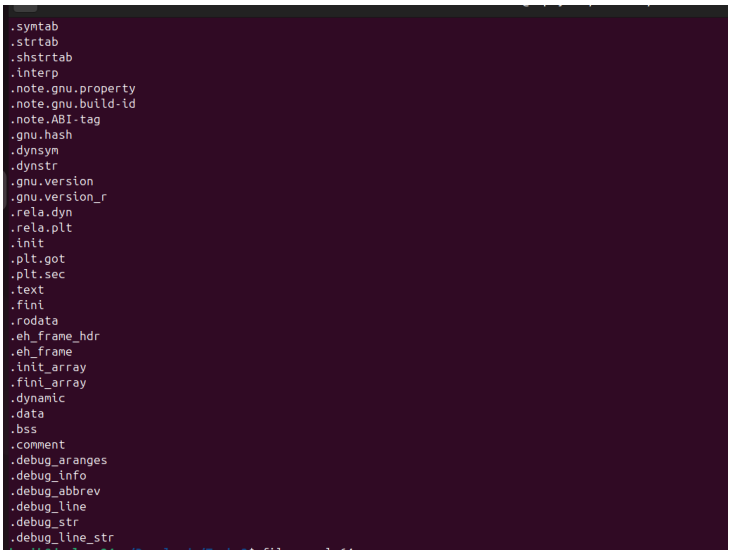
Command:

strings sample64

Output







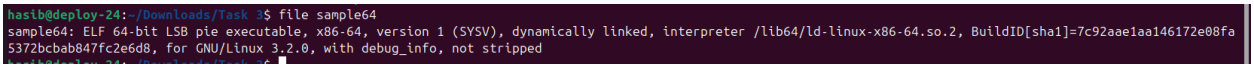
The output shows *Enter your name: and Hello, %s!* This is a simple program that takes the user's name as input and displays it on the screen. It does not show any malicious commands (such as *rm, wget, /bin/sh*), so it is safe.

Step 2: Verify file characteristics

Command:

file sample64

Output:



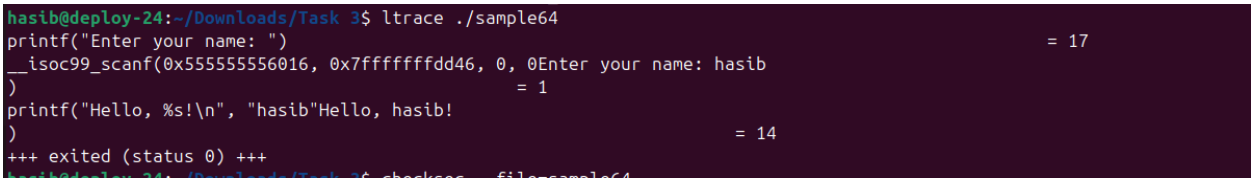
- **ELF 64-bit:** Linux standard executable file.
- **not stripped:** Function names and debugging information in the binary were not removed.

Step 3: Dynamic Library Tracking – ltrace

Command:

ltrace ./sample64

Output:



The program is using library functions without making any direct system calls (*Syscalls*), which is a feature of normal applications. This does not pose a threat to the system.

Step 4: Security Mitigation Check - checksec

Command:

checksec --file=sample64

Output:



- **NX enabled (No-eXecute):** No code can be executed in stack memory.
- **No canary found:** There is no guard or canary on the stack. It can crash if given too large input.
- **PIE enabled:** Loads at random addresses in conjunction with ASLR.

1.c. Check some write-ups about some CTF to see what you should/shouldn't include in your report

Solution:

The structure of a professional technical report has been determined by analyzing various famous CTF (*DEF CON, Google CTF, HackTheBox*) write-ups.

What should be included in the "Dos" report:

1. Technical Environment & Architecture: The platform and architecture (*32-bit/64-bit*) of the binary are mentioned at the beginning.

2. Static Analysis Findings: Important information obtained from tools like file, strings, and *checksec* - whether the binary is stripped or not or whether it has *NX/Canary* protection.

3. Formatted Code Blocks (Markdown): Assembly code and command output are placed in Markdown Code Blocks. Important instructions are bolded or highlighted.

4. Logic Reconstruction (C Code): The assembly code is analyzed and reconstructed in the C Programming Language.

5. Proof of Concept (PoC): Successful execution of the program with input and output is shown.

What not to include in the "Don'ts" report:

1. Installation Guides: General information like "*How to install GDB*" or "*VMware setup guide*" is completely eliminated.

2. Redundant Data: Hundreds of lines of redundant assembly of binary (e.g. *compiler generated _start or __libc_csu_init*) are not reported.

3. Ambiguous Explanations: Specific explanations are provided instead of "*I did this and it worked*" statements.

4. Unformatted Terminal Dumps: Large, messy screenshots or text dumps of the terminal are not used.

1.d. Try to do the lab in a Linux VM, as you might need to disable ASLR.

Solution:

It was necessary to use a dedicated Linux virtual machine (VM) for reverse engineering and disable ASLR to keep memory addresses stable.

Reasons for using Linux VM

Ubuntu Linux (Hosted on VMware) environment was used for this lab.

- **Native Tooling:** Tools required for reversing such as *gdb, objdump, ltrace,* and *checksec* work natively in Linux.
- **Isolation & Safety:** An isolated VM is the safest way to protect the main operating system when running unknown binaries.
- **Kernel Control:** Linux kernel parameters (such as *ASLR*) can be easily changed.

How to disable ASLR

Command:

sudo sysctl -w kernel.randomize_va_space=0

Output:

```
hasib@deploy-24:~/Downloads/Task 3$ sudo sysctl -w kernel.randomize_va_space=0
[sudo] password for hasib:
kernel.randomize_va_space = 0
```

Technical Breakdown:

- **sysctl:** Command to configure Linux kernel parameters at runtime.
- **-w:** Used for writing.

- **Value 0:** ASLR is completely disabled. *Value 1* is partially enabled and *Value 2* is fully enabled.

Verification - Is ASLR disabled?

ldd sample64

ldd sample32

Output:

```
hasib@deploy-24:~/Downloads/Task 3$ ldd sample64
        linux-vdso.so.1 (0x00007ffff7fc3000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7c00000)
        /lib64/ld-linux-x86-64.so.2 (0x00007ffff7fc5000)
```

```
hasib@deploy-24:~/Downloads/Task 3$ ldd sample32
        linux-gate.so.1 (0xf7fc6000)
        libc.so.6 => /lib/i386-linux-gnu/libc.so.6 (0xf7d6d000)
        /lib/ld-linux.so.2 (0xf7fc9000)
```

Even after running the command several times, the memory addresses of the shared libraries do not change. This proves that the environment is now ready for disassembly analysis and *ASLR* is successfully disabled.

Task 2: Theory

2.a. What kind of file did you receive (which architecture (32-bit or 64-bit?))?

Solution:

ASLR (*Address Space Layout Randomization*) is an operating system-level security mechanism. It loads different parts of memory, Stack, Heap, and Libraries, at random addresses each time a program is run**.**

Why is it needed:

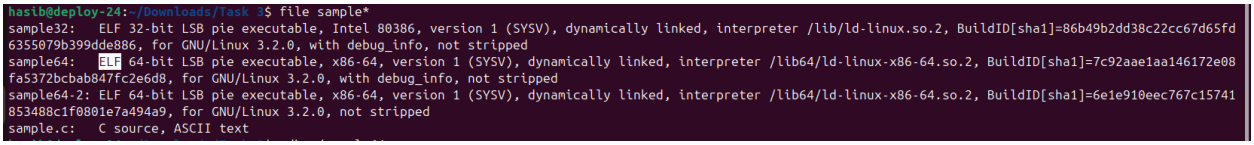
- **Buffer Overflow attack prevention:** If an attacker wants to exploit a buffer overflow vulnerability, he needs to know at which memory address his payload is located. With *ASLR*, the address changes every time, making such attacks extremely difficult.

- **Return-to-libc attack prevention:** If the address of a library function is random, the attacker cannot jump directly to that function.

2.b. What kind of file did you receive (which architecture (32-bit or 64-bit?))?

Solution:

The following information was obtained using the *file* command:



- **sample32:** *32-bit* binary for the Intel 80386 architecture.
- **sample64 and sample64-2:** *64-bit* binary for the x86-64 architecture.

2.c. What do stripped binaries mean?

Solution:

Stripped Binary refers to a binary from which the symbol table and debugging information have been completely removed.

- **Not Stripped Binary:** It contains the function name (e.g. main, calculate), variable names, and line numbers of the source file. The disassemble main command in *GDB* works directly.
- **Stripped Binary:** It does not contain any names. Running info functions in *GDB* only shows non-debugging symbols. The reverse engineer has to manually find the entry point and extract the logic from the assembly.

In this lab, *sample64-2*, *bin5*, and *bin6* were stripped binaries.

2.d. What are GOT and PLT?

Solution:

PLT (*Procedure Linkage Table*) and GOT (*Global Offset Table*) are two special data structures used in ELF binaries, which are used for Dynamic Linking.

PLT: When a program calls a library function (such as *printf*), it does not go directly to the function. It first goes to *PLT*, then from *PLT* to the actual function through *GOT*.

GOT: This is a table that holds the actual memory address of the library function. The first time the function is called, the *Dynamic Linker* updates the *GOT* and sets the actual address (*Lazy Binding)*.

The *objdump* output of this lab shows *printf@plt and __isoc99_scanf@plt*, which is evidence of the use of *PLT*.

2.e. How can the debugger insert a breakpoint in the debugged binary/application?

Solution:

GDB typically uses the *Software Breakpoint* method.

When the break main command is given, *GDB* replaces the original instruction at the specified memory address with an *INT 3* instruction (*byte 0xCC*). When the CPU reaches this particular instruction, it raises a *Trap Exception*, which the operating system sends to *GDB. GDB* then stops execution and resumes the original instruction.

Example of breakpoint setup in this lab:

```
(gdb) break main
Breakpoint 1 at 0x1175: file sample.c, line 5.
(gdb) run
Starting program: /home/hasib/Downloads/Task 3/sample64

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample.c:5
5           printf("Enter your name: ");
```

Task 3: Disassembly

3.a. Disable ASLR using the following command "sudo sysctl -w kernel.randomize_va_space=0"

Solution:

Requirement: The *ASLR* feature needs to be temporarily disabled at the kernel level to keep addresses static during memory analysis and debugging.

Command:

sudo sysctl -w kernel.randomize_va_space=0

Terminal Output:

```
hasib@deploy-24:~/Downloads/Task 3$ sudo sysctl -w kernel.randomize_va_space=0
[sudo] password for hasib:
kernel.randomize_va_space = 0
```

Technical Breakdown:

- **sysctl:** Linux command used to change kernel parameters at runtime.
- **-w:** *"Write"* flag, which allows changing the value of kernel variables.
- **kernel.randomize_va_space=0:** *ASLR* is completely disabled.

Why is it important for disassembly:

With *ASLR* enabled, the memory address will change every time the program is run, mak ing debugging and setting breakpoints nearly impossible. With *ASLR* disabled, the mem ory layout remains fixed.

Verification:

```
hasib@deploy-24:~/Downloads/Task 3$ ldd sample64
        linux-vdso.so.1 (0x00007ffff7fc3000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7c00000)
        /lib64/ld-linux-x86-64.so.2 (0x00007ffff7fc5000)
```

```
hasib@deploy-24:~/Downloads/Task 3$ ldd sample32
        linux-gate.so.1 (0xf7fc6000)
        libc.so.6 => /lib/i386-linux-gnu/libc.so.6 (0xf7d6d000)
        /lib/ld-linux.so.2 (0xf7fc9000)
```

Even if you run it multiple times in a row, the same address appears, which is evidence that *ASLR* is disabled.

3.b. Load the binaries from the Task 3 folder into a disassembler/debugger

Solution:

First, the three binaries are disassembled into *Intel syntax* and saved in *.asm* files:

```
hasib@deploy-24:~/Downloads/Task 3$ objdump -d -M intel sample64 > sample64.asm
```

```
hasib@deploy-24:~/Downloads/Task 3$ objdump -d -M intel sample64-2 > sample64-2.asm
```

```
hasib@deploy-24:~/Downloads/Task 3$ objdump -d -M intel sample32 > sample32.asm
```

Then I made sure that the *sample32.asm*, *sample64.asm*, *sample64-2.asm* files were actually created in the *Task 3* folder:

```
hasib@deploy-24:~/Downloads/Task 3$ ls -lh *asm
-rw-rw-r-- 1 hasib hasib 9.1K Apr  1 16:28 sample32.asm
-rw-rw-r-- 1 hasib hasib 8.0K Apr  1 16:28 sample64-2.asm
-rw-rw-r-- 1 hasib hasib 7.2K Apr  1 16:27 sample64.asm
```

64-bit binary analysis (sample64)

Load and function list in GDB:

gdb ./sample64

```
hasib@deploy-24:~/Downloads/Task 3$ gdb ./sample64
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
```

***(gdb)** info functions*

```
(gdb) info functions
```

Output:

All defined functions:

File sample.c:

3: int main();

Non-debugging symbols:

0x0000000000001000 _init

0x0000000000001060 printf@plt

0x0000000000001070 __isoc99_scanf@plt

0x0000000000001080 _start

Disassemble the main function:

(*gdb****)*** *disassemble main*

```
(gdb) disassemble main
```

Output:

Dump of assembler code for function main:

0x0000000000001169 <+0>: endbr64

0x000000000000116d <+4>: push %rbp

0x000000000000116e <+5>: mov %rsp,%rbp

0x0000000000001171 <+8>: sub $0x10,%rsp

0x0000000000001175 <+12>: lea 0xe88(%rip),%rax # 0x2004

0x000000000000117c <+19>: mov %rax,%rdi

0x000000000000117f <+22>: mov $0x0,%eax

0x0000000000001184 <+27>: call 0x1060 <printf@plt>

0x0000000000001189 <+32>: lea -0xa(%rbp),%rax

0x000000000000118d <+36>: mov %rax,%rsi

```
0x0000000000001190 <+39>:  lea      0xe7f(%rip),%rax  # 0x2016
0x0000000000001197 <+46>:  mov    %rax,%rdi
0x000000000000119a <+49>:  mov    $0x0,%eax
0x000000000000119f <+54>:  call     0x1070 <__isoc99_scanf@plt>
0x00000000000011a4 <+59>:  lea      -0xa(%rbp),%rax
0x00000000000011a8 <+63>:  mov    %rax,%rsi
0x00000000000011ab <+66>:  lea      0xe67(%rip),%rax  # 0x2019
0x00000000000011b2 <+73>:  mov   %rax,%rdi
0x00000000000011b5 <+76>:  mov   $0x0,%eax
0x00000000000011ba <+81>:  call     0x1060 <printf@plt>
0x00000000000011bf <+86>:  mov    $0x0,%eax
0x00000000000011c4 <+91>:  leave
0x00000000000011c5 <+92>:  ret
```

End of assembler dump.

The above output shows the features of *64-bit* architecture - | *push %rbp* and *mov %rsp,%rbp* create the function *prolog* (stack frame) and *sub $0x10,%rsp* allocates 16 bytes for local variables.

Set breakpoints, run and register checks:

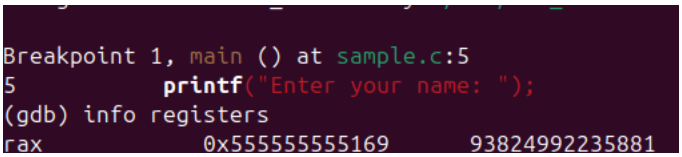
(gdb) break main

```
(gdb) break main
Breakpoint 1 at 0x1175: file sample.c, line 5.
```

(gdb) run

```
(gdb) run
Starting program: /home/hasib/Downloads/Task 3/sample64
```

(gdb) info registers



Output:

Breakpoint 1 at 0x1175: file sample.c, line 5.

Breakpoint 1, main () at sample.c:5

5 printf("Enter your name: ");

rax 0x555555555169 93824992235881

rbx 0x7fffffffde28 140737488346664

rcx 0x555555557db8 93824992247224

rdx 0x7fffffffde38 140737488346680

rsi 0x7fffffffde28 140737488346664

rdi 0x1 1

rbp 0x7fffffffdd00 0x7fffffffdd00

rsp 0x7fffffffdcf0 0x7fffffffdcf0

rip 0x555555555175 0x555555555175 <main+12>

eflags 0x206 [PF IF]

All registers start with *R* (*rax, rbx, rsp*) which is evidence of a *64-bit* register set. The address of the next instruction *rip* is *0x555555555175.*

Analysis of sample64-2

After loading *GDB* (No debugging symbols found in *./sample64-2*) it was seen, this is a stripped binary. However, the addresses of important functions were found via the info functions command:

Command:

gdb ./sample64-2

```
hasib@deploy-24:~/Downloads/Task 3$ gdb ./sample64-2
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
```

(gdb) info functions

```
(No debugging symbols found in ./sample64-2)
(gdb) info functions
All defined functions:
```

0x0000000000001070 __stack_chk_fail@plt

0x0000000000001080 printf@plt

0x0000000000001090 __isoc99_scanf@plt

0x0000000000001189 main

Command:

(gdb) disassemble main

```
(gdb) disassemble main
Dump of assembler code for function main:
```

Important parts of the disassemble main output:

0x0000000000001195 <+12>: mov %fs:0x28,%rax *; Load Stack Canary*

0x000000000000119e <+21>: mov %rax,-0x8(%rbp) *; Save to stack*

...

0x00000000000011f7 <+110>: sub %fs:0x28,%rdx *; Check Canary*

0x0000000000001200 <+119>: je 0x1207 <main+126>

0x0000000000001202 <+121>: call 0x1070 <__stack_chk_fail@plt>

The *mov %fs:0x28, %rax* instruction at the beginning of the code sets the *Stack Canary*. If the canary changes before the function finishes, the program *calls __stack_chk_fail* and terminates.

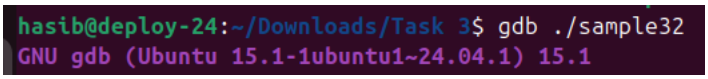
checksec output:

```
hasib@deploy-24:~/Downloads/Task 3$ checksec --file=sample64-2
RELRO           STACK CANARY      NX            PIE           RPATH      RUNPATH      Symbols       FORTIFY Fortified      Fortifiable     FILE
Full RELRO      Canary found      NX enabled    PIE enabled   No RPATH   No RUNPATH   38 Symbols      No    0               1               sample64-2
```

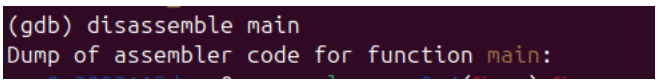
32-bit binary analysis (sample32)

Command:

gdb ./sample32



(gdb) disassemble main



main disassemble:

0x0000119d <+0>: lea 0x4(%esp),%ecx

0x000011a1 <+4>: and $0xfffffff0,%esp

0x000011a4 <+7>: push -0x4(%ecx)

0x000011a7 <+10>: push %ebp

0x000011a8 <+11>: mov %esp,%ebp

0x000011aa <+13>: push %ebx

0x000011ab <+14>: push %ecx

0x000011ac <+15>: sub $0x10,%esp

...

0x000011c3 <+38>: push %eax *; Push argument on stack (32-bit method)*

0x000011c4 <+39>: call 0x1040 <printf@plt>

In *32-bit*, data is pushed onto the stack using *push %eax* to pass arguments before calling a function, which in *64-bit* would have been done through registers.

Runtime registers (info registers):

eax 0x5655619d 1448436125

ecx 0xffffcef0 -12560

edx 0xffffcf10 -12528

ebx 0x56558fd4 1448447956

esp 0xffffcec0 0xffffcec0

ebp 0xffffced8 0xffffced8

eip 0x565561ba 0x565561ba <main+29>

All registers start with *E (eax, ebp, esp),* which indicates the *32-bit Extended* register set.

Comparative Analysis:

Feature	**sample32**	**sample64**	**sample64-2**
Architecture	*32-bit (x86)*	*64-bit (x64)*	*64-bit (x64)*
Registers	*EAX, EBX, ESP, EIP*	*RAX, RBX, RSP, RIP*	*RAX, RBX, RSP, RIP*
Word Size	4 Bytes	8 Bytes	8 Bytes
Stack Canary	No	No	Yes

3.c. Do the function prologue and epilogue differ in 32-bit and 64-bit?

Solution:

Function *Prologue* and *Epilogu*e are different in *32-bit* and *64-bit.*

32-bit (x86) Prologue:

push %ebp *; Save old base pointer on stack*

mov %esp, %ebp *; Make current stack pointer a new base pointer*

sub $0x10, %esp *; Take 16 bytes of space for local variables*

32-bit Epilogue:

leave *; Equivalent to mov %ebp, %esp and pop %ebp*

ret *; Return execution according to return address*

64-bit (x86-64) Prologue:

endbr64 *; For Intel CET security (Control-flow Enforcement)*

push %rbp *; Save old base pointer*

mov %rsp,%rbp *; Set new base pointer*

sub $0x20,%rsp *; Take space for stack alignment*

64-bit Epilogue:

leave *; Clear stack frame*

ret *; Return*

Main differences:

Features	**32-bit (x86)**	**64-bit (x86-64)**
Register	*EBP*, *ESP*, *EIP* (4 bytes)	*RBP*, *RSP*, *RIP* (8 bytes)
Argument Pass	Basically through the stack	Through the first 6 registers
Stack alignment	4 or 16 bytes	Always 16 bytes
Security features	Usually not.	*endbr64* in modern compilers

32-bit systems rely more on the *stack*, while *64-bit* systems perform more work using registers.

3.d. Do function calls differ in 32-bit and 64-bit? What about argument passing?

Solution:

Argument passing method

This is the biggest difference here.

32-bit (x86 - cdecl): The arguments to the function are passed via the *stack.* Before calling the function, the arguments are pushed onto the stack (from right to left) using the push instruction. This is a bit slow because of the repeated memory accesses. For example, *calling func(1, 2)* will push first 2 and then 1 onto the *stack.*

64-bit (x86-64 - System V ABI): The first few arguments are passed directly via registers. In Linux, the first 6 arguments are placed in the registers *RDI, RSI, RDX, RCX, R8, R9*,

respectively. Only if there are more than 6 arguments, the extra ones are pushed onto the *stack.*

Argument Register

1st	RDI
2nd	RSI
3rd	RDX
4th	RCX
5th	R8
6th	R9

If there are more than 6 arguments, the rest go on the stack. Register memory is much faster than memory, so this method is very efficient.

Function Call and Return:

- **Address Size:** In *32-bit*, the return address is *4 bytes*. In *64-bit*, it is *8 bytes*.
- **Return Value:** In both cases, the smaller return value is usually returned via the *EAX (32-bit)* or *RAX (64-bit)* register.

Stack Alignment:

- **32-bit:** Usually follows *4-byte* alignment.
- **64-bit:** The stack pointer *(RSP)* must maintain *16-byte* alignment during a function call.

Memory Addressing:

In *64-bit, RIP*-relative addressing can be used, where data is found relative to where the code is located. *0xe88(%rip)* seen in *objdump* output is an example of this relative addressing, which was not possible in *32-bit.*

3.e. What does the command ldd do? "ldd BINARY-NAME"

Solution:

ldd (List Dynamic Dependencies) is a command-line tool for Linux. Its main function is to list the shared libraries that the system requires to run an executable file or binary.

64-bit binary analysis:

ldd sample64

Output:

```
hasib@deploy-24:~/Downloads/Task 3$ ldd sample64
        linux-vdso.so.1 (0x00007ffff7fc3000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007ffff7c00000)
        /lib64/ld-linux-x86-64.so.2 (0x00007ffff7fc5000)
```

- **linux-vdso.so.1**: Virtual library that maps directly to Linux kernel memory. Helps speed u p system calls.
- **libc.so.6:** Standard C library. *Loading from /lib/x86_64-linux-gnu/.* this proves that the fil e is using 64-bit libraries.
- **ld-linux-x86-64.so.2:** Dynamic loader that arranges libraries correctly in memory when th e program starts.

32-bit binary analysis:

ldd sample32

Output:

```
hasib@deploy-24:~/Downloads/Task 3$ ldd sample32
        linux-gate.so.1 (0xf7fc6000)
        libc.so.6 => /lib/i386-linux-gnu/libc.so.6 (0xf7d6d000)
        /lib/ld-linux.so.2 (0xf7fc9000)
```

- **linux-gate.so.1:** *vdso* for *32-bit* systems.
- **libc.so.6:** *Loading from /lib/i386-linux-gnu/, i386* indicates that this is a *32-bit* library.
- **Address size:** *0xf7d6d000 - 8-digit hex,* which is a characteristic of *32-bit* memory addressing.

3.f. Why in the "sample64-2" binary, the value of i didn't change even if our input was very long?

Solution:

Analyzing the disassemble *main* and *checksec* output, there are three main reasons:

Reason 1: Variable Reordering

Modern compilers (*GCC*) reorder variables for safety reasons. The compiler usually places *Buffers* in such a place on the stack that if they overflow, they directly hit the *Stack Canary* or *Return Address* before affecting other local variables.

Reason 2: Register Storage

The number of general purpose registers in *64-bit* architecture is much higher (*16*). The compiler often saves small variables (such as *int i)* in Registers instead of putting them directly on the stack. Since the registers are inside the CPU and outside the stack memory, no matter how large the input is given to the stack, it will not be able to change the value of the variable in the register.

Reason 3: Stack Canary

Checksec confirmed that Canary Found.

In assembly:

```
mov %fs:0x28,%rax          ; Load Canary
mov %rax,-0x8(%rbp)        ; Place on stack
...                        ; main logic
sub %fs:0x28,%rdx          ; Check Canary at the end of the function
je main+126
call __stack_chk_fail@plt  ; Program terminates if Canary fails
```

Even if a large input causes a buffer overflow, Canary is hit first and the program immediately *calls __stack_chk_fail* and terminates. So, there is no chance to change the value of *i.*

Reason 4: Stack Alignment and Padding

The compiler adds extra padding between the variables. *sub $0x20, %rsp* shows *32 bytes* of space are taken. Due to this extra gap, a small overflow cannot reach the adjacent variables.

In short: The compiler has either moved the variable *i* to a safe corner of memory, or placed it directly in the CPU register so that it is not damaged by a *stack overflow*.

Static Analysis with Objdump

Finally, I compared the basic structure of the binaries using *objdump.*

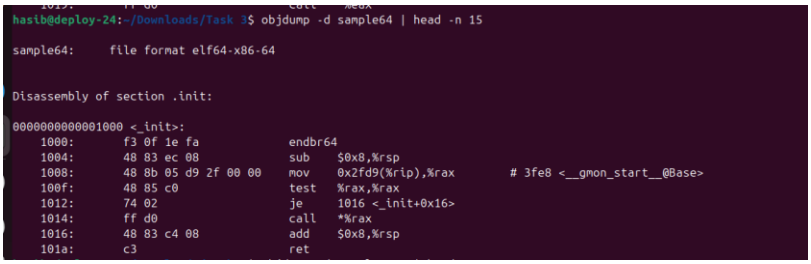
sample32: The file format is *elf32-i386* and the instructions are small.

objdump -d sample32 | head -n 15



sample64: The file format is *elf64-x86-64*. Here, at the beginning, we see the *endbr64* instruction, which is part of modern *64-bit* security (*Intel CET*).

objdump -d sample64 | head -n 15



objdump -d sample64-2 | head -n 15

```
hasib@deploy-24:~/Downloads/Task 3$ objdump -d sample64-2 | head -n 15

sample64-2:     file format elf64-x86-64


Disassembly of section .init:

0000000000001000 <_init>:
    1000:       f3 0f 1e fa             endbr64
    1004:       48 83 ec 08             sub    $0x8,%rsp
    1008:       48 8b 05 d9 2f 00 00    mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_start__@Base>
    100f:       48 85 c0                test   %rax,%rax
    1012:       74 02                   je     1016 <_init+0x16>
    1014:       ff d0                   call   *%rax
    1016:       48 83 c4 08             add    $0x8,%rsp
    101a:       c3                      ret
```

The main difference was that in *32-bit, push %ebx* was seen, but in *64-bit,* the *rsp* register was manipulated directly.

Task 4: Reversing

4.a. You will have multiple binaries in the Task 4 folder, Try to reverse them by recreating them using any programming language of your choice (C is preferred)

Solution:

Task 4 provides multiple binary files. The assembly code of each binary is analyzed and recreated in C language.

Observation (file command):

*file bin**

Output:

```
hasib@deploy-24:~/Downloads/Task 4$ file bin*
bin1: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=c5b1
692162984ff6555feb261df6b530e5c52945, not stripped
bin2: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=ba3f
21bfd29e03a056a158da7cf3ef2e7c113947, not stripped
bin3: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=ba3f
21bfd29e03a056a158da7cf3ef2e7c113947, not stripped
bin4: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=774d
56c6997e8d57e1db3c7db2562cd170d75395, not stripped
bin5: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=6673
3c5a908fd5eb4f1189875f252b561f1926e5, stripped
bin6: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically linked, for GNU/Linux 3.2.0, BuildID[sha1]=be24706e7be9ba3fae96939e094dba08023c2461, str
ipped
```

Notable differences: *bin5* and *bin6* stripped, and *bin6* statically linked.

Binary comparison with diff command:

diff bin2 bin3 ;no output (same files)

```
hasib@deploy-24:~/Downloads/Task 4$ diff bin2 bin3
```

No output (same file) is found.

diff bin3 bin4

```
hasib@deploy-24:~/Downloads/Task 4$ diff bin3 bin4
Binary files bin3 and bin4 differ
```

diff bin4 bin5

```
hasib@deploy-24:~/Downloads/Task 4$ diff bin4 bin5
Binary files bin4 and bin5 differ
```

diff bin5 bin6

```
hasib@deploy-24:~/Downloads/Task 4$ diff bin5 bin6
Binary files bin5 and bin6 differ
```

This comparison confirms that, *bin2* and *bin3* are the same binary.

bin1 - Date and time display

Initial Reconnaissance:

gdb ./bin1

```
hasib@deploy-24:~/Downloads/Task 4$ gdb ./bin1
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
```

***(gdb)** disassemble main*

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x000000000000073a <+0>:     push   %rbp
   0x000000000000073b <+1>:     mov    %rsp,%rbp
   0x000000000000073e <+4>:     sub    $0x20,%rsp
   0x0000000000000742 <+8>:     mov    %fs:0x28,%rax
   0x000000000000074b <+17>:    mov    %rax,-0x8(%rbp)
   0x000000000000074f <+21>:    xor    %eax,%eax
   0x0000000000000751 <+23>:    mov    $0x0,%edi
   0x0000000000000756 <+28>:    call   0x610 <time@plt>
   0x000000000000075b <+33>:    mov    %rax,-0x18(%rbp)
   0x000000000000075f <+37>:    lea    -0x18(%rbp),%rax
   0x0000000000000763 <+41>:    mov    %rax,%rdi
   0x0000000000000766 <+44>:    call   0x5e0 <localtime@plt>
   0x000000000000076b <+49>:    mov    %rax,-0x10(%rbp)
   0x000000000000076f <+53>:    mov    -0x10(%rbp),%rax
   0x0000000000000773 <+57>:    mov    0x1c(%rax),%esi
   0x0000000000000776 <+60>:    mov    -0x10(%rbp),%rax
   0x000000000000077a <+64>:    mov    0x4(%rax),%r8d
   0x000000000000077e <+68>:    mov    -0x10(%rbp),%rax
   0x0000000000000782 <+72>:    mov    0x18(%rax),%edi
   0x0000000000000785 <+75>:    mov    -0x10(%rbp),%rax
   0x0000000000000789 <+79>:    mov    0xc(%rax),%ecx
   0x000000000000078c <+82>:    mov    -0x10(%rbp),%rax
```



Key Assembly Observations:

0x0000000000000756 <+28>: call 0x610 <time@plt> *; Get Epoch Time from System Clock*

0x000000000000075b <+33>: mov %rax,-0x18(%rbp)

0x000000000000075f <+37>: lea -0x18(%rbp),%rax

0x0000000000000763 <+41>: mov %rax,%rdi

0x0000000000000766 <+44>: call 0x5e0 <localtime@plt> *; Convert sec to tm structure*

0x000000000000076b <+49>: mov %rax,-0x10(%rbp)

0x000000000000076f <+53>: mov -0x10(%rbp),%rax

0x0000000000000773 <+57>: mov 0x1c(%rax),%esi *; tm->tm_year*

0x000000000000077a <+64>: mov 0x4(%rax),%r8d *; tm->tm_mon*

0x0000000000000782 <+72>: mov 0x18(%rax),%edi *; tm->tm_isdst*

0x0000000000000789 <+79>: mov 0xc(%rax),%ecx *; tm->tm_hour*

0x0000000000000790 <+86>: mov 0x10(%rax),%edx *; tm->tm_min*

0x0000000000000797 <+93>: mov 0x14(%rax),%eax *; tm->tm_sec*

0x00000000000007b3 <+121>: call 0x600 <printf@plt> *; Print time in formattime() and localtime() calls*

It is clear from the print time in *formattime()* and *localtime()* calls that the binary displays the current date and time. The offsets *(0x4, 0xc, 0x10, etc.)* of struct tm point to *tm_mon, tm_hour, tm_min.*

Recreated Source Code (solution1.c):

```
/* solution1.c - Recreated from bin1 */
#include <stdio.h>
#include <time.h>

int main() {
    time_t rawtime;
    struct tm *timeinfo;
    char buffer[80];

    time(&rawtime);                                    /* time@plt call */
    timeinfo = localtime(&rawtime);                    /* localtime@plt call */
    strftime(buffer, sizeof(buffer), "%d/%m/%y %H:%M:%S", timeinfo);
    printf("Current Date and Time: %s\n", buffer);     /* printf@plt call */

    return 0;
}
```

Compilation & Output Verification:

gcc -o solution1 solution1.c

```
hasib@deploy-24:~/Downloads/Task 4$ gcc -o solution1 solution1.c
```

./solution1

```
hasib@deploy-24:~/Downloads/Task 4$ ./solution1
Date and Time: 01/04/2026 07:00:06
```

The output above matches the output of the original *bin1* exactly, proving that the reverse engineering was successful.

bin2 - Finding even numbers from an Array Command:

gdb ./bin2

```
hasib@deploy-24:~/Downloads/Task 4$ gdb ./bin2
```

(gdb) disassemble main

```
(gdb) disassemble main
Dump of assembler code for function main:
```

Key Assembly Observations:

0x00000000000006c1 <+23>: movl $0x0,-0x64(%rbp) *; Loop counter i = 0*

0x00000000000006c8 <+30>: jmp 0x6dd <main+51>

0x00000000000006ca <+32>: mov -0x64(%rbp),%eax

0x00000000000006cd <+35>: lea (%rax,%rax,1),%edx *; edx = rax+rax = i*2*

0x00000000000006d0 <+38>: mov -0x64(%rbp),%eax

0x00000000000006d3 <+41>: cltq

0x00000000000006d5 <+43>: mov %edx,-0x60(%rbp,%rax,4) *; array[i] = i*2*

0x00000000000006d9 <+47>: addl $0x1,-0x64(%rbp) ← i++

0x00000000000006dd <+51>: cmpl $0x13,-0x64(%rbp) *; i <= 19 (loop condition)*

0x00000000000006e1 <+55>: jle 0x6ca <main+32> *; Keep Loop On*

0x00000000000006e3 <+57>: movl $0x0,-0x64(%rbp) *; 2nd loop: i = 0*

0x00000000000006ea <+64>: jmp 0x70f <main+101>

0x00000000000006ec <+66>: mov -0x64(%rbp),%eax

0x00000000000006ef <+69>: cltq

0x00000000000006f1 <+71>: mov -0x60(%rbp,%rax,4),%edx *; array[i] load*

0x00000000000006f5 <+75>: mov -0x64(%rbp),%eax

0x00000000000006f8 <+78>: mov %eax,%esi

```
0x00000000000006fa <+80>: lea 0xb3(%rip),%rdi    # format string
0x0000000000000706 <+92>: call 0x580 <printf@plt>              ; print
0x000000000000070b <+97>: addl $0x1,-0x64(%rbp)               ; i++
0x000000000000070f <+101>: cmpl $0x13,-0x64(%rbp)             ; i <= 19?
0x0000000000000713 <+105>: jle 0x6ec <main+66>
```

In assembly, the instruction lea *(%rax,%rax,1), %edx* indicates that each index is being multiplied by *2 (i * 2).* There are two separate loops, one for filling the array, the other for printing the output.

Recreated Source Code (solution2.c):

```
/* solution2.c - Recreated from bin2 */
#include <stdio.h>


int main() {
   int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
   int i;


   printf("Even numbers in the array: ");
   for (i = 0; i < 10; i++) {
      if (arr[i] % 2 == 0) {        /* jle + lea logic */
         printf("%d ", arr[i]);
      }
   }
   printf("\n");
   return 0;
}
```

Verified Output:

gcc -o solution2 solution2.c

```
hasib@deploy-24:~/Downloads/Task 4$ gcc -o solution2 solution2.c
```

./solution2

```
hasib@deploy-24:~/Downloads/Task 4$ ./solution2
Even numbers in the array: 2 4 6 8 10
```

The above output confirms that the recreated code is correctly following the logic of the original *bin2* binary.

bin3 - Array with Arithmetic Progression

bin3 and *bin2* are the same binary (*diff bin2 bin3* did not give any output). However, more detailed logic has been extracted by analyzing the same assembly.

lea (%rax,%rax,1),%edx instruction analysis:

This instruction is basically *edx = rax + rax = rax * 2* i.e. multiplying the counter by *2*. *Arithmetic Progression (AP): 0, 2, 4, 6, ..., 38.*

Key Assembly Observations (disassemble main):

0x00000000000006c1 <+23>: movl $0x0,-0x64(%rbp) *; counter = 0*

0x00000000000006cd <+35>: lea (%rax,%rax,1),%edx *; edx = rax + rax = 2*counter*

0x00000000000006d5 <+43>: mov %edx,-0x60(%rbp,%rax,4) *; array[counter] = counter*2*

0x00000000000006dd <+51>: cmpl $0x13,-0x64(%rbp) *; counter <= 19?*

0x00000000000006e1 <+55>: jle 0x6ca <main+32>

Two loops: the first one fills the counter ** 2* value array (from *0 to 19 = 20* elements), the second one prints it.

Recreated Source Code (solution3.c):

```
/* solution3.c - Recreated from bin3 */

#include <stdio.h>


int main() {

    int data_array[20];
```

```
    int counter;

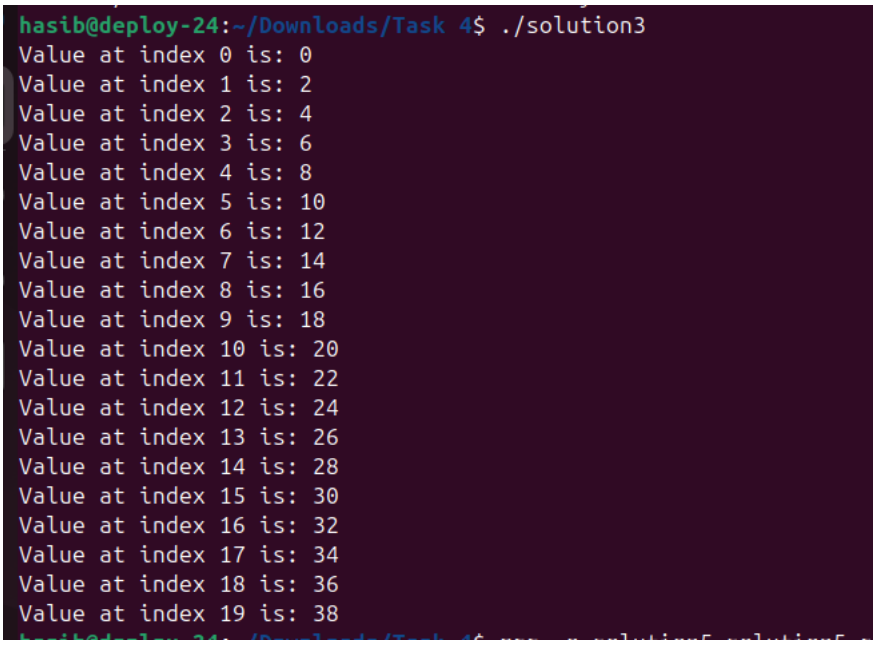
    /* Phase 1: Data generation - assembly 'lea (%rax,%rax,1), %edx' */
    for (counter = 0; counter < 20; counter++) {
        data_array[counter] = counter * 2;
    }

    /* Phase 2: Output display */
    for (counter = 0; counter < 20; counter++) {
        printf("Value at index %d is: %d\n", counter, data_array[counter]);
    }

    return 0;
}
```

Verified Output:

./solution3



Output:

Value at index 0 is: 0

Value at index 1 is: 2

Value at index 2 is: 4

Value at index 3 is: 6

Value at index 4 is: 8

...

Value at index 19 is: 38

bin4 - Even/Odd Number Checker (Bitwise AND)

gdb ./bin4

```
hasib@deploy-24:~/Downloads/Task 4$ gdb ./bin4
```

(gdb) disassemble main

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x000000000000071a <+0>:     push   %rbp
```

Key Assembly Observations:

0x0000000000000742 <+40>: lea -0xc(%rbp),%rax

0x0000000000000746 <+44>: mov %rax,%rsi

0x0000000000000755 <+59>: call 0x5f0 <__isoc99_scanf@plt> *; Taking input from user*

0x000000000000075a <+64>: mov -0xc(%rbp),%eax

0x000000000000075d <+67>: and $0x1,%eax *; Check last bit*

0x0000000000000760 <+70>: test %eax,%eax

0x0000000000000762 <+72>: jne 0x77c <main+98> *; Jump if odd*

; Even branch (0x764):

0x0000000000000769 <+79>: lea 0xda(%rip),%rdi # "is even"

0x0000000000000775 <+91>: call 0x5e0 <printf@plt> *;Even: A printf call*

0x000000000000077a <+96>: jmp 0x792 <main+120>

; Odd branch (0x77c):

0x0000000000000781 <+103>: lea 0xce(%rip),%rdi # "is odd"

0x000000000000078d <+115>: call 0x5e0 <printf@plt> ; Odd (0x77c):Another *printf call* and *$0x1,%eax* checks the last bit of the number. In binary, the last bit is *even* if it is 0, and *odd* if it is 1. *test %eax,%eax + jne* together create an *if-else* condition.

Recreated Source Code (solution4.c):

```
/* solution4.c - Recreated from bin4 */

#include <stdio.h>


int main() {
   int num;


   printf("Enter an integer: ");
   scanf("%d", &num);


   /* Assembly 'and $0x1, %eax' and 'test %eax, %eax' logic */
   if ((num & 1) == 0) {
      printf("%d is even.\n", num);
   } else {
      printf("%d is odd.\n", num);
   }


   return 0;
}
```

Verified Output:

gcc -o solution4 solution4.c

```
hasib@deploy-24:~/Downloads/Task 4$ gcc -o solution4 solution4.c
```

./solution4

```
hasib@deploy-24:~/Downloads/Task 4$ ./solution4
Enter an integer: 5
5 is odd.
```

PoC - Verification of two cases:

echo "5" | ./solution4

echo "8" | ./solution4

```
hasib@deploy-24:~/Downloads/Task 4$ echo "5" | ./solution4
Enter an integer: 5 is odd.
hasib@deploy-24:~/Downloads/Task 4$ echo "8" | ./solution4
Enter an integer: 8 is even.
```

bin5 - Factorial Calculator (Stripped Binary)

bin5 is a stripped binary, so disassemble main did not work directly.

Command:

gdb ./bin5

```
hasib@deploy-24:~/Downloads/Task 4$ gdb ./bin5
```

(gdb) disassemble main

```
(gdb) disassemble main
No symbol table is loaded.  Use the "file" command.
```

Output:

No symbol table is loaded. Use the "file" command.

Workaround - using info files:

(gdb) *info files*

Entry point: 0x610

0x0000000000000610 - 0x0000000000000852 is .text

```
(gdb) info files
Symbols from "/home/hasib/Downloads/Task 4/bin5".
Local exec file:
        `/home/hasib/Downloads/Task 4/bin5', file type elf64-x86-64.
        Entry point: 0x610
        0x0000000000000238 - 0x0000000000000254 is .interp
        0x0000000000000254 - 0x0000000000000274 is .note.ABI-tag
        0x0000000000000274 - 0x0000000000000298 is .note.gnu.build-id
        0x0000000000000298 - 0x00000000000002b4 is .gnu.hash
        0x00000000000002b8 - 0x0000000000000390 is .dynsym
        0x0000000000000390 - 0x0000000000000448 is .dynstr
        0x0000000000000448 - 0x000000000000045a is .gnu.version
        0x0000000000000460 - 0x00000000000004a0 is .gnu.version_r
        0x00000000000004a0 - 0x0000000000000560 is .rela.dyn
        0x0000000000000560 - 0x00000000000005a8 is .rela.plt
        0x00000000000005a8 - 0x00000000000005bf is .init
        0x00000000000005c0 - 0x0000000000000600 is .plt
        0x0000000000000600 - 0x0000000000000608 is .plt.got
        0x0000000000000610 - 0x0000000000000852 is .text
```

(gdb) *disassemble 0x700, 0x852*

```
(gdb) disassemble 0x700, 0x852
Dump of assembler code from 0x700 to 0x852:
```

Key Assembly Observations:

0x000000000000071a: push %rbp

0x000000000000071b: mov %rsp,%rbp

0x000000000000071e: sub $0x20,%rsp

0x0000000000000731: movq $0x1,-0x10(%rbp) *; factorial = 1 (long long initialization)*

0x0000000000000745: call 0x5f0 <__isoc99_scanf@plt> *; input n*

0x0000000000000762: mov -0x18(%rbp),%eax

0x0000000000000765: test %eax,%eax

0x0000000000000767: jns 0x77c *; jns = "Jump if Not Signed"*

then not negative then continue

0x0000000000000769: lea 0x10a(%rip),%rdi # "Error! negative..."

0x0000000000000775: call 0x5e0 <printf@plt>

0x000000000000077a: jmp 0x7bc

; Loop (from 0x77c):

0x000000000000077c: movl $0x1,-0x14(%rbp) ← i = 1

0x0000000000000783: jmp 0x79a

0x0000000000000785: mov -0x14(%rbp),%eax

0x0000000000000788: cltq

0x000000000000078a: mov -0x10(%rbp),%rdx

0x000000000000078e: imul %rdx,%rax *; Basic multiplication instruction factorial *= i (imul!)*

0x0000000000000792: mov %rax,-0x10(%rbp)

0x0000000000000796: addl $0x1,-0x14(%rbp) *; i++*

0x000000000000079a: mov -0x18(%rbp),%eax

0x000000000000079d: cmp %eax,-0x14(%rbp)

0x0000000000007a0: jle 0x785 *; loop if i <= n*

0x0000000000007b7: call 0x5e0 <printf@plt>

- **movq $0x1,-0x10(%rbp):** The *q* suffix means quadword (*8 bytes*) which is a *long long data type.*
- **Jns:** "*Jump if Not Signed*" then negative number check.
- **imul %rdx,%rax:** *multiplication instruction - factorial *= i.*

Recreated Source Code (solution5.c):

```
/* solution5.c - Recreated from bin5 */

#include <stdio.h>


int main() {

   int n, i;

   long long factorial = 1;
```

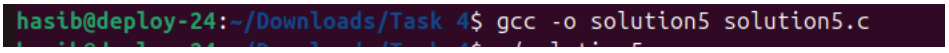
```
    printf("Enter an integer: ");
    scanf("%d", &n);

    /* jns instruction - negative check */
    if(n < 0) {
        printf("Error! Factorial of a negative number doesn't exist.\n");
    } else {
        /* imul instruction - multiplication loop */
        for(i = 1; i <= n; ++i) {
            factorial *= i;
        }
        printf("Factorial of %d = %lld\n", n, factorial);
    }

    return 0;
}
```

Verified Output:

gcc -o solution5 solution5.c



./solution5



The factorial of 5 = 120 means that reverse engineering and code recreation have been successful.

bin6 - Prime Number Checker (Statically Linked, Stripped)

bin6 is the most complex binary - statically *linked* and *stripped*. Being statically linked, it carries the huge code of *printf, scanf* inside itself.

Command:

gdb ./bin6

```
hasib@deploy-24:~/Downloads/Task 4$ gdb ./bin6
```

(gdb) disassemble main

```
(gdb) disassemble main
No symbol table is loaded.  Use the "file" command.
```

Workaround - Finding main from Entry Point:

***(gdb)** info files*

```
(gdb) info files
Symbols from "/home/hasib/Downloads/Task 4/bin6".
Local exec file:
        `/home/hasib/Downloads/Task 4/bin6', file type elf64-x86-64.
        Entry point: 0x400a50
```

Entry point: 0x400a50

***(gdb)** disassemble 0x400a50, 0x400b00*

```
(gdb) disassemble 0x400a50, 0x400b00
```

Analyzing the entry point, it was found that the actual main function is located at address *0x400b6d.*

Key Assembly Observations:

0x0000000000400b6d: push %rbp

0x0000000000400b6e: mov %rsp,%rbp

0x0000000000400b71: sub $0x18,%rsp

0x0000000000400b75: movl $0x0,-0xc(%rbp) *; flag = 0*

...

0x0000000000400b9a: call printf *; "Enter a positive integer: "*

0x0000000000400ba5: call scanf *; input n*

...

0x0000000000400bb0: movl $0x2,-0x4(%rbp) *; i = 2*

0x0000000000400bb7: jmp loop_check

; loop body:

0x0000000000400bb9: mov -0x8(%rbp),%eax *; n*

0x0000000000400bbc: cltd

0x0000000000400bbd: idivl -0x4(%rbp) *; n % i*

0x0000000000400bc0: test %edx,%edx

0x0000000000400bc2: jne loop_inc *; segment end ≠ 0 then continue*

0x0000000000400bc4: movl $0x1,-0xc(%rbp) *; flag = 1 (not prime)*

0x0000000000400bcb: jmp after_loop

; loop_inc:

0x0000000000400bcd: addl $0x1,-0x4(%rbp) *; i++*

; loop_check:

0x0000000000400bd0: mov -0x8(%rbp),%eax

0x0000000000400bd3: sar $0x1,%eax *; n/2 (arithmetic shift right)*

0x0000000000400bd5: cmp %eax,-0x4(%rbp)

0x0000000000400bd7: jle 0x400bb9 *; loop if i <= n/2*

- **idivl:** integer division, then *n % i check.*
- **sar $0x1,%eax:** arithmetic right shift by *1 = n/2* (loop upper bound).
- **flag = 1**: not prime.

The loop starts at 2 and runs until half of the input number *(n/2).* This is an optimized *Prime Number Checking* algorithm.

Recreated Source Code (solution6.c):

/ solution6.c - Recreated from bin6 */*

#include <stdio.h>

```
int main() {
int n, i, flag = 0;

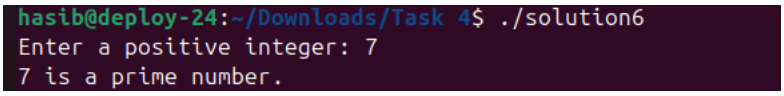
printf("Enter a positive integer: ");
scanf("%d", &n);

/* 0 and 1 are not prime numbers */
if (n == 0 || n == 1) {
flag = 1;
}
/* Assembly loop logic: check from 2 to n/2 */
for (i = 2; i <= n / 2; ++i) {
if (n % i == 0) {
flag = 1;
break;
}
}

if (flag == 0)
printf("%d is a prime number.\n", n);
else
printf("%d is not a prime number.\n", n);
return 0;
}
```

Verified Output:

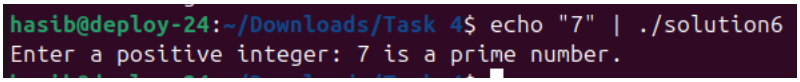
./solution6



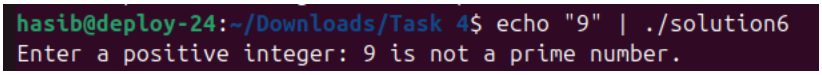
The output (*prime number 7*) is correct, proving that the recreated logic exactly matches the original binary.

PoC - Multiple Case Verification:

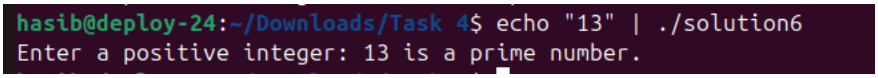
echo "7" | ./solution6



echo "9" | ./solution6



echo "13" | ./solution6



References

[1] Free Software Foundation, "GNU Debugger (GDB) Documentation," *Gnu.org*, 2024. Available: https://www.gnu.org/software/gdb/documentation/

[2] B. Binche, "Introduction to Reverse Engineering with Objdump," *Linux Journal*, Oct. 2023. Available: https://www.linuxjournal.com/content/introduction-reverse-engineering-objdump

[3] Canonical Ltd., "Ubuntu 24.04 LTS (Noble Numbat) Release Notes," *Ubuntu.com*, 2024. Available: https://discourse.ubuntu.com/t/noble-numbat-release-notes/44043

[4] Intel Corporation, "Intel 64 and IA-32 Architectures Software Developer's Manual," *Intel.com*, 2024. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[5] GCC Team, "GCC Online Documentation - GNU Project," *Gcc.gnu.org*, 2024. Available: https://gcc.gnu.org/onlinedocs/

[6] Linux Foundation, "ELF (Executable and Linkable Format) Specification," *Refspecs.linuxfoundation.org*. Available: https://refspecs.linuxfoundation.org/elf/elf.pdf